HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BỘ MÔN PYTHON



BÀI TẬP LỚN

HỌC PHẦN: LẬP TRÌNH PYTHON

Đề tài: Kindred – nền tảng kết nối cộng đồng đồng điệu với ghép đôi TF-IDF, trò chuyện thời gian thực và gọi WebRTC

Giảng viên hướng dẫn: Phạm Danh Tuyên

Lớp học phần: Nhóm 10

Nhóm thực hiện: 1

Danh sách thành viên:

Đinh Tuấn Quang - B24DCKH111

Lê Anh Tú - B24DCKH128

Nguyễn Duy Long - B24DCKH076

HÀ NỘI, THÁNG 11/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 4
1.1. Bối cảnh và bài toán 4
1.2. Mục tiêu 4
1.3. Phạm vi và dữ liệu 4
1.4. Phân công công việc 5
1.5. Khai báo sử dụng công cụ AI 5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 6
2.1. TF-IDF và độ đo cosine 6
2.2. REST API và JSON Web Token 6
2.3. WebSocket 6
2.4. WebRTC 6
2.5. ORM và kiến trúc phân tầng 6
CHƯƠNG 3. KIẾN TRÚC TỔNG THỂ 7
3.1. Các thành phần 7
3.2. Cấu trúc mã nguồn 7
3.3. Luồng xử lý một yêu cầu REST 8
CHƯƠNG 4. CƠ SỞ DỮ LIỆU 10
CHƯƠNG 5. BACKEND VÀ API 12
5.1. Xác thực và phân quyền 12
5.2. Danh sách endpoint 12
5.3. Quy tắc nghiệp vụ nổi bật 13
CHƯƠNG 6. THUẬT TOÁN GHÉP ĐÔI 14
6.1. Ý tưởng 14
6.2. Quy trình tính điểm 14
6.3. Ví dụ kết quả 15
CHƯƠNG 7. NHẮN TIN THỜI GIAN THỰC 16
7.1. Xác thực kết nối WebSocket 16
7.2. Quản lý kết nối 16
7.3. Dự phòng bằng REST 16
CHƯƠNG 8. GỌI THOẠI VÀ GỌI VIDEO WEBRTC 17
8.1. Luồng báo hiệu 17
8.2. Các cơ chế đảm bảo 17
8.3. Giao diện cuộc gọi 18
CHƯƠNG 9. BẢNG TIN, THÔNG BÁO VÀ KIỂM DUYỆT 19
9.1. Bảng tin cộng đồng 19
9.2. Thông báo và trạng thái hoạt động 19
9.3. Chặn, báo cáo và bảng điều khiển quản trị 19
CHƯƠNG 10. GIAO DIỆN REACT 20
10.1. Các chức năng chính 20
10.2. Cách giao diện tương tác với backend 23

CHƯƠNG 11. KIỂM THỬ VÀ TRIỂN KHAI 24
11.1. Kiểm thử tự động 24
11.2. Triển khai 24
CHƯƠNG 12. CÔNG NGHỆ VÀ KẾT LUẬN 26
12.1. Công nghệ và thư viện 26
12.2. Kết quả đạt được 26
12.3. Hạn chế 26
12.4. Hướng phát triển 27
TÀI LIỆU THAM KHẢO 28

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và bài toán

Các mạng xã hội phổ biến được thiết kế quanh việc giữ chân người dùng bằng bảng tin vô tận và số lượt tương tác. Với nhiều người, đặc biệt là sinh viên và người mới đi làm, điều họ cần lại là tìm được những người có cùng hoàn cảnh, cùng cảm xúc và cùng mục tiêu để chia sẻ. Các nền tảng hiện có gặp một số hạn chế khi đáp ứng nhu cầu này.

- Gợi ý kết bạn chủ yếu dựa trên danh bạ hoặc bạn chung, không dựa trên hoàn cảnh và trạng thái tâm lý của người dùng.
- Hồ sơ chỉ gồm thông tin hành chính nên khó mở đầu cuộc trò chuyện, người dùng không biết mình và người kia giống nhau ở điểm nào.
- Trò chuyện và gọi điện thường tách rời các tính năng khám phá, người dùng phải chuyển sang ứng dụng khác.
- Thiếu cơ chế an toàn đủ gọn (chặn, báo cáo, kiểm duyệt) cho một cộng đồng nhỏ mới hình thành.

Đề tài **Kindred** (khẩu hiệu “Find people who understand you”) xây dựng một ứng dụng web giải quyết các vấn đề trên: người dùng điền một bảng khảo sát ngắn, hệ thống ghép đôi bằng kỹ thuật xử lý ngôn ngữ truyền thống (TF-IDF và độ đo cosine), giải thích lý do đồng điệu, sau đó cho phép kết nối, nhắn tin thời gian thực và gọi thoại hoặc video trực tiếp giữa hai trình duyệt.

1.2. Mục tiêu

- Xây dựng quy trình khảo sát onboarding gồm 6 nhóm nội dung và thuật toán ghép đôi có giải thích (điểm tương đồng, sở thích chung, lý do đồng điệu).
- Xây dựng backend FastAPI theo kiến trúc phân tầng router, service, model; xác thực bằng JWT và Google OAuth 2.0.
- Xây dựng kênh nhắn tin thời gian thực bằng WebSocket và gọi thoại, gọi video 1-1 bằng WebRTC với báo hiệu đi qua chính kết nối WebSocket.
- Xây dựng bảng tin cộng đồng, thông báo, chặn hai chiều, báo cáo vi phạm và bảng điều khiển quản trị.
- Có bộ kiểm thử tự động cho backend và triển khai thực tế trên ba nền tảng đám mây (Vercel, Render, Supabase).

1.3. Phạm vi và dữ liệu

Hệ thống là một ứng dụng web nhiều người dùng, gồm giao diện React một trang (SPA) và backend REST kèm WebSocket. Toàn bộ mã nguồn khoảng 18 nghìn dòng (Python, JavaScript, CSS), gồm 11 router, 10 module service, 13 bảng dữ liệu và 11

trang giao diện. Trong báo cáo này, các ảnh chụp giao diện được lấy khi chạy ứng dụng với bộ dữ liệu mẫu do script `seed_data.py` tạo ra (6 tài khoản mẫu cùng bài viết, lượt thích và bình luận), không phải dữ liệu của người dùng thật. Đề tài tập trung vào kết nối giữa người với người; không bao gồm chuyên gia tư vấn hay trợ lý AI.

1.4. Phân công công việc

Bảng 1. Phân công công việc của nhóm

Thành viên	Phụ trách
Đinh Tuấn Quang	Thuật toán ghép đôi, xác thực và bảo mật, nhắn tin thời gian thực, gọi WebRTC, thuyết trình
Nguyễn Duy Long	Cơ sở dữ liệu, bảng tin, thông báo, chặn và báo cáo, quản trị, kiểm thử, triển khai, làm slide
Lê Anh Tú	Giao diện người dùng React, luồng khảo sát, viết báo cáo

1.5. Khai báo sử dụng công cụ AI

Trong quá trình thực hiện bài tập, nhóm có sử dụng công cụ AI là trợ lý lập trình Claude Code (Anthropic). Công cụ này được dùng cho các nhiệm vụ sau:

- Khảo sát mã nguồn của dự án để tổng hợp kiến trúc, luồng xử lý, danh sách endpoint và danh sách thư viện phục vụ viết báo cáo.
- Vẽ các sơ đồ minh hoạ (kiến trúc, quan hệ cơ sở dữ liệu, quy trình ghép đôi, luồng cuộc gọi, xác thực WebSocket).
- Chạy ứng dụng với dữ liệu mẫu, chụp ảnh màn hình giao diện và chạy lại các bộ kiểm thử để lấy kết quả đưa vào báo cáo.

Các nhiệm vụ trên đều do thành viên nhóm giao và kiểm tra lại. Các kết quả kiểm thử trong chương 11 là kết quả chạy thực tế các script có trong repo. Ngoài ra, nhóm cũng dùng trợ lý AI trong quá trình viết mã dự án theo bộ quy tắc ghi trong tệp `AI_RULES.md`; các quy tắc này cố định công nghệ sử dụng và yêu cầu thuật toán ghép đôi không dùng học sâu hay mô hình ngôn ngữ lớn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. TF-IDF và độ đo cosine

TF-IDF (Term Frequency – Inverse Document Frequency) biểu diễn một văn bản thành vector, trong đó mỗi từ có trọng số bằng tần suất xuất hiện trong văn bản nhân với nghịch đảo tần suất xuất hiện trong toàn bộ tập văn bản. Từ phổ biến ở mọi hồ sơ bị giảm trọng số, từ đặc trưng của một số ít người được tăng trọng số. Thư viện scikit-learn dùng công thức làm mịn: idf(t) = ln((1 + n) / (1 + df(t))) + 1.

Độ tương đồng giữa hai vector u và v được đo bằng cosine của góc giữa chúng: cos(u, v) = (u · v) / (‖u‖ · ‖v‖). Vì các vector TF-IDF không âm nên giá trị nằm trong khoảng [0, 1], phù hợp để quy đổi thành phần trăm.

2.2. REST API và JSON Web Token

Backend cung cấp REST API qua HTTP. Sau khi đăng nhập, máy chủ cấp một JSON Web Token (JWT) ký bằng thuật toán HS256, chứa mã người dùng (`sub`), thời điểm cấp (`iat`) và thời hạn (`exp`). Mỗi request sau đó gửi token trong header `Authorization: Bearer`; máy chủ giải mã, kiểm tra chữ ký và thời hạn mà không cần lưu phiên. Mật khẩu được băm bằng bcrypt với salt ngẫu nhiên.

2.3. WebSocket

WebSocket (RFC 6455) là kênh hai chiều duy trì trên một kết nối TCP, cho phép máy chủ chủ động đẩy dữ liệu tới trình duyệt. Dự án dùng WebSocket để phát tin nhắn tức thì, cập nhật trạng thái trực tuyến và làm kênh báo hiệu cho cuộc gọi.

2.4. WebRTC

WebRTC cho phép hai trình duyệt truyền âm thanh và hình ảnh trực tiếp ngang hàng (P2P). Để thiết lập kết nối, hai bên cần trao đổi SDP (mô tả phiên: offer và answer) và các ứng viên ICE (địa chỉ mạng khả dụng). WebRTC không quy định kênh trao đổi này nên ứng dụng phải tự xây dựng kênh báo hiệu (signaling). Máy chủ STUN giúp mỗi trình duyệt biết địa chỉ công khai của mình để vượt NAT. Luồng media sau đó đi thẳng giữa hai trình duyệt, không qua backend.

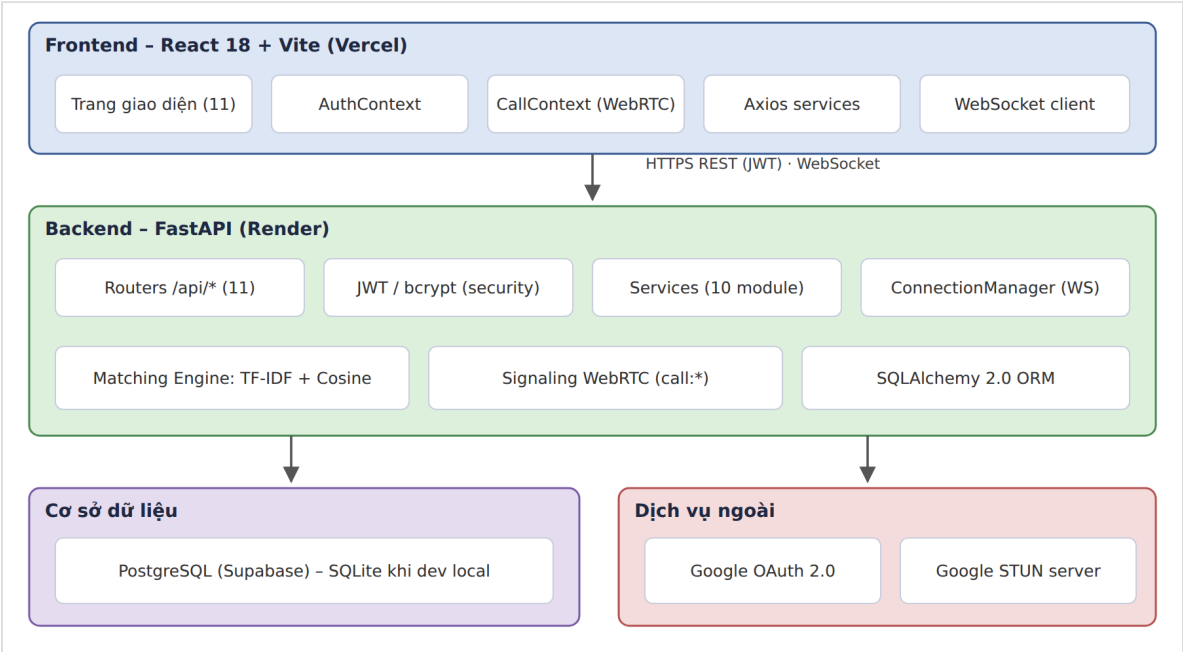
2.5. ORM và kiến trúc phân tầng

SQLAlchemy 2.0 ánh xạ bảng cơ sở dữ liệu thành lớp Python, giúp cùng một mã chạy được trên SQLite (phát triển) và PostgreSQL (sản xuất). Backend tách ba tầng: router nhận và kiểm tra request bằng schema Pydantic, service chứa nghiệp vụ, model mô tả dữ liệu. Cách tách này giúp kiểm thử nghiệp vụ mà không cần dựng HTTP.

CHƯƠNG 3. KIẾN TRÚC TỔNG THỂ

3.1. Các thành phần

Hệ thống chia thành ba lớp chính. Lớp giao diện là ứng dụng React chạy trên Vercel. Lớp backend là dịch vụ FastAPI chạy trên Render, gồm các router REST, tầng service, bộ máy ghép đôi và bộ quản lý kết nối WebSocket. Lớp lưu trữ là PostgreSQL trên Supabase (SQLite khi phát triển cục bộ). Hai dịch vụ bên ngoài được dùng là Google OAuth 2.0 để đăng nhập và máy chủ STUN công khai của Google cho WebRTC.



Hình 1. Kiến trúc tổng thể của hệ thống

Điểm đáng chú ý là kênh báo hiệu cuộc gọi dùng lại kết nối WebSocket toàn cục của người dùng (`/api/ws/user`), nên người nhận nhận được cuộc gọi dù đang ở bất kỳ trang nào của ứng dụng. Luồng âm thanh và hình ảnh đi thẳng giữa hai trình duyệt nên backend không chịu tải media.

3.2. Cấu trúc mã nguồn

Cấu trúc thư mục chính của dự án:

```
caithunhat/
├── backend/
│   ├── app/
│   │   ├── core/        # câú hình, JWT, băm mật khẩủ
│   │   ├── db/          # engine, session, Base
│   │   ├── models/      # 9 tệp ORM (13 bảng)
│   │   ├── schemas/     # Pydantic request/response
│   │   ├── services/    # nghiệp vụ (10 module)
│   │   ├── routers/     # REST + WebSocket (11 router)
│   │   ├── websocket/   # ConnectionManager
│   │   └── main.py      # khởi tạo app, CORS, tài khoản admin
│   ├── tests/           # 8 script kiểm thử
│   ├── seed_data.py     # dữ liệu mâũ
│   └── requirements.txt
├── frontend/
│   └── src/
│       ├── pages/       # 11 trang
│       ├── components/  # CallModal, Navbar, ReportModal...
│       ├── contexts/    # AuthContext, CallContext
│       └── services/    # Axios client và các service API
├── vercel.json          # rewrite SPA
└── README.md, MVP_STATUS.md, AI_RULES.md
```

Bảng 2. Các phần chính của dự án

Phần	Nội dung
Xác thực và bảo mật	`core/security.py`: băm bcrypt, tạo và giải mã JWT; `routers/deps.py`: lấy người dùng hiện tại, kiểm tra quyền admin
Ghép đôi	`services/matching_service.py`: TF-IDF, cosine, điểm thưởng, lý do đồng điệu
Kết nối và trò chuyện	`connection_service.py`, `chat_service.py`, `routers/chat.py` (REST và hai endpoint WebSocket)
Thời gian thực	`websocket/manager.py`: quản lý socket theo phòng và theo người dùng, trạng thái online
Bảng tin và thông báo	`post_service.py`, `notification_service.py`
An toàn và quản trị	`report_service.py`, `admin_service.py`, mô hình `Block` và `Report`
Giao diện	React 18, 11 trang, `CallContext` giữ kết nối WebSocket toàn cục và phiên gọi

3.3. Luồng xử lý một yêu cầu REST

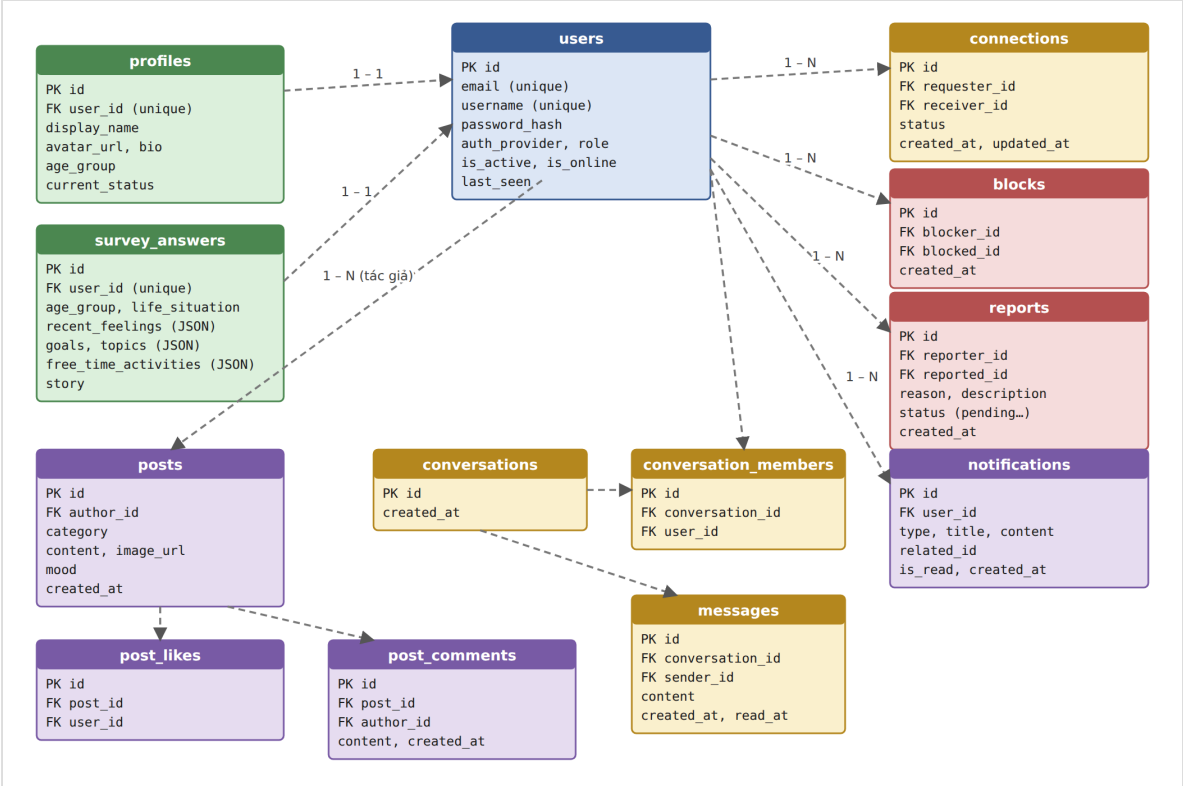
1. Trình duyệt gọi API qua Axios; interceptor tự gắn JWT từ `localStorage` vào header.
2. Router xác thực token qua dependency, kiểm tra dữ liệu vào bằng schema Pydantic.
3. Router gọi hàm trong tầng service để xử lý nghiệp vụ và truy cập cơ sở dữ liệu qua SQLAlchemy.

4. Service có thể tạo thông báo cho người liên quan; kết quả được chuyển thành schema phản hồi và trả về dạng JSON.
5. Nếu token hết hạn hoặc không hợp lệ (HTTP 401), giao diện xoá token và chuyển về trang đăng nhập.

CHƯƠNG 4. CƠ SỞ DỮ LIỆU

Phần do Nguyễn Duy Long phụ trách.

Dữ liệu quan hệ được truy cập qua SQLAlchemy 2.0. Môi trường sản xuất dùng PostgreSQL của Supabase với `NullPool` và `pool_pre_ping` để tránh giữ kết nối chết; môi trường phát triển dùng SQLite. Có 13 bảng, quan hệ giữa chúng được mô tả ở Hình 2.



Hình 2. Sơ đồ quan hệ thực thể (ERD)

Bảng 3. Các nhóm bảng chính

Nhóm	Nội dung
Tài khoản	`users` (email, username, mật khẩu băm, vai trò, trạng thái online, `last_seen`), `profiles` (tên hiển thị, ảnh, tiểu sử)
Khảo sát	`survey_answers`: nhóm tuổi, hoàn cảnh, cảm xúc, mục tiêu, chủ đề, hoạt động rảnh rỗi (các trường danh sách lưu kiểu JSON), câu chuyện cá nhân
Kết nối	`connections` (trạng thái pending, accepted, rejected), `blocks` (chặn hai chiều)
Trò chuyện	`conversations`, `conversation_members`, `messages` (có `read_at` để theo dõi đã đọc)
Bảng tin	`posts` (danh mục, nội dung, ảnh, tâm trạng), `post_likes`, `post_comments`

Kiểm duyệt và thông báo	`reports` (lý do, mô tả, trạng thái xử lý), `notifications` (loại, nội dung, `is_read`)

Các trường danh sách trong khảo sát được lưu bằng kiểu JSON nên cùng một mô hình chạy được trên cả SQLite và PostgreSQL mà không cần bảng phụ. Bảng được tạo tự động khi ứng dụng khởi động (`Base.metadata.create_all`); với SQLite, hàm khởi tạo còn kiểm tra và bổ sung các cột mới của bảng `users` để không làm mất dữ liệu cũ. Khi khởi động, hệ thống cũng tạo tài khoản quản trị mặc định nếu chưa có, email và mật khẩu đọc từ biến môi trường.

CHƯƠNG 5. BACKEND VÀ API

Phần do Đinh Tuấn Quang phụ trách.

5.1. Xác thực và phân quyền

- **Đăng ký, đăng nhập**: định danh bằng email hoặc tên người dùng; mật khẩu băm bcrypt, không lưu dạng rõ.
- **Google OAuth 2.0**: giao diện nhận ID token từ Google Identity Services, backend xác minh bằng thư viện `google-auth` rồi cấp JWT của hệ thống. Tài khoản tạo bằng Google có `auth_provider = google`.
- **JWT HS256**: token chứa `sub`, `iat`, `exp`; thời hạn mặc định 60 phút, cấu hình qua biến môi trường. Hàm giải mã trả về rỗng nếu token sai chữ ký hoặc hết hạn.
- **Phân quyền**: dependency kiểm tra người dùng đang hoạt động; các endpoint `/api/admin/*` yêu cầu `role = admin`. Hồ sơ công khai không bao giờ trả về email hay mã băm mật khẩu.
- **CORS**: chỉ cho phép danh sách origin cụ thể (localhost, tên miền Vercel của dự án và các origin khai báo qua biến môi trường), kèm biểu thức chính quy giới hạn dạng `https://*.vercel.app`.

5.2. Danh sách endpoint

Bảng 4. Các nhóm endpoint của backend

Nhóm	Endpoint tiêu biểu	Yêu cầu
Xác thực, người dùng	`POST /api/auth/register`, `/login`, `/google`; `GET /api/users/me`, `/api/users/{id}/public`	Công khai (đăng ký, đăng nhập); còn lại cần JWT
Khảo sát, hồ sơ	`POST /api/survey`, `GET /api/survey/me`; `GET/PUT /api/profile/me`	JWT
Ghép đôi	`GET /api/matches`	JWT, phải có khảo sát
Kết nối	`GET /api/connections`, `/requests`; `POST /{user_id}`; `PUT /{id}/accept`, `/reject`	JWT
Trò chuyện	`GET /api/conversations`; `POST /api/conversations/{user_id}`; `GET/POST .../messages`; `PUT /api/messages/{id}/read`	JWT
Thời gian thực	`WS /api/ws/chat/{conversation_id}?token=`; `WS /api/ws/user?token=`	JWT trong query

Bảng tin	`GET/POST /api/posts`; `GET/DELETE /api/posts/{id}`; `POST .../like`, `.../comments`	JWT; xoá: tác giả hoặc admin
An toàn	`POST /api/reports`; `POST/DELETE /api/blocks/{user_id}`; `GET /api/blocks`	JWT
Quản trị	`GET /api/admin/stats`, `/users`, `/reports`; `PUT .../toggle-status`, `/reports/{id}`	Vai trò admin
Thông báo	`GET /api/notifications`; `PUT .../{id}/read`, `/read-all`	JWT

Tài liệu tương tác của toàn bộ API được FastAPI sinh tự động tại `/docs` (Swagger UI), giúp nhóm thử endpoint mà không cần công cụ ngoài.

5.3. Quy tắc nghiệp vụ nổi bật

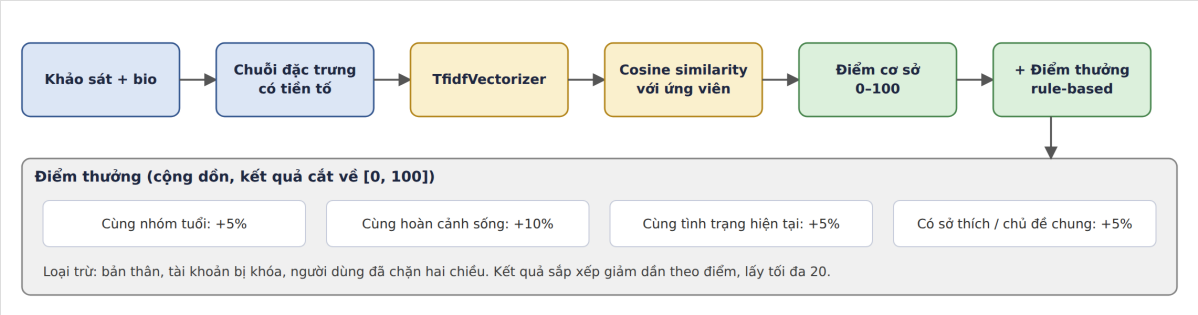
- Không thể gửi lời mời kết nối cho chính mình, cho người đã là bạn, cho người đã gửi lời mời hoặc khi giữa hai người có quan hệ chặn.
- Chỉ người nhận lời mời mới được chấp nhận hoặc từ chối; người khác gọi sẽ nhận lỗi 403.
- Chỉ thành viên của một cuộc hội thoại mới được đọc lịch sử hoặc gửi tin vào đó (403 với người ngoài); tạo hội thoại 1-1 là thao tác idempotent.
- Khi chặn một người, hội thoại giữa hai bên bị khoá, nút gọi bị vô hiệu hoá và người đó biến mất khỏi danh sách khám phá.
- Báo cáo được chuyển cho admin; admin có thể phê duyệt (tuỳ chọn khoá luôn tài khoản bị tố cáo) hoặc bác bỏ.

CHƯƠNG 6. THUẬT TOÁN GHÉP ĐÔI

Phần do Đinh Tuấn Quang phụ trách.

6.1. Ý tưởng

Thuật toán nằm trong `services/matching_service.py` và là bộ máy lai: lõi là TF-IDF cùng độ đo cosine trên văn bản đặc trưng của từng người, sau đó cộng điểm thưởng theo các quy tắc dễ giải thích. Không sử dụng học sâu hay mô hình ngôn ngữ lớn, nên kết quả tái lập được và không phụ thuộc dịch vụ ngoài.



Hình 3. Quy trình tính điểm ghép đôi

6.2. Quy trình tính điểm

1. **Biểu diễn văn bản**: mỗi câu trả lời khảo sát được gắn tiền tố ngữ nghĩa để các giá trị cùng tên ở hai trường khác nhau không bị trộn lẫn, ví dụ `tuổi_21–24`, `hoàn_cảnh_…`, `cảm_xúc_…`, `mục_tiêu_…`, `chủ_đề_…`, `hoạt_động_…`; sau đó nối với phần tự thuật, lời nhắn và tiểu sử, rồi chuyển thành chữ thường.
2. **Vector hoá**: `TfidfVectorizer` được huấn luyện trên tập gồm văn bản của người đang xem và của mọi ứng viên hợp lệ.
3. **Cosine**: tính độ tương đồng giữa vector của người đang xem và từng ứng viên; điểm cơ sở = cosine × 100, làm tròn.
4. **Điểm thưởng**: cộng dồn theo bảng 5, tổng điểm cắt về đoạn [0, 100].
5. **Giải thích**: sinh danh sách sở thích chung (giao của chủ đề và hoạt động) và các lý do đồng điệu bằng quy tắc, hiển thị ngay trên thẻ người dùng.

Bảng 5. Điểm thưởng theo quy tắc

Điều kiện	Điểm cộng
Cùng nhóm độ tuổi	+5
Cùng hoàn cảnh sống (`life_situation`)	+10
Cùng tình trạng hiện tại (`current_status`)	+5
Có ít nhất một chủ đề hoặc sở thích chung	+5

Các ứng viên bị loại gồm chính người đang xem, tài khoản bị khoá và những người có quan hệ chặn với người đang xem (theo cả hai chiều). Kết quả được sắp xếp giảm dần theo điểm và trả về tối đa 20 người. Người dùng chưa làm khảo sát nhận phản hồi `has_survey = false` để giao diện hiển thị lời mời làm khảo sát.

6.3. Ví dụ kết quả

Với bộ dữ liệu mẫu, khi tài khoản `mai_linh_ui` mở trang Khám phá, hệ thống trả về ba người đầu danh sách như bảng 6. Cột lý do lấy trực tiếp từ giao diện (Hình 9).

Bảng 6. Kết quả ghép đôi trên dữ liệu mẫu

Người dùng	Điểm	Lý do đồng điệu hiển thị
Lê Hà My	59%	Cùng nhóm tuổi (21–24); chung 5 chủ đề và sở thích; mục tiêu đồng điệu
Đỗ Thu Trang	53%	Cùng tình trạng hiện tại (đang đi làm); chung 5 chủ đề và sở thích; mục tiêu đồng điệu
Vũ Đức Minh	35%	Cùng tình trạng hiện tại (đang đi làm); chung 1 chủ đề; mục tiêu đồng điệu

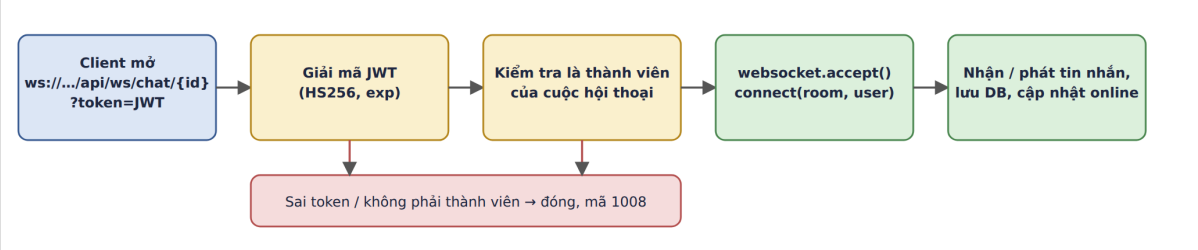
Điểm số có ý nghĩa xếp hạng tương đối giữa các ứng viên chứ không phải xác suất hợp nhau. Vì TF-IDF được huấn luyện lại trên tập ứng viên ở mỗi lần gọi nên cùng một cặp người có thể có điểm cơ sở hơi khác khi số lượng người dùng thay đổi; điểm thưởng thì cố định.

CHƯƠNG 7. NHẮN TIN THỜI GIAN THỰC

Phần do Đinh Tuấn Quang phụ trách.

7.1. Xác thực kết nối WebSocket

Trình duyệt không gắn được header tuỳ ý khi mở WebSocket nên JWT được gửi qua tham số truy vấn `token`. Máy chủ xác thực hai lớp trước khi gọi `websocket.accept()`: giải mã token, rồi kiểm tra trong cơ sở dữ liệu rằng người dùng là thành viên của cuộc hội thoại. Nếu không đạt, kết nối bị đóng với mã 1008 (vi phạm chính sách).



Hình 4. Xác thực kết nối WebSocket phòng chat

7.2. Quản lý kết nối

Lớp `ConnectionManager` (một thể hiện dùng chung) giữ hai cấu trúc trong bộ nhớ: `active_connections[conversation_id][user_id]` là tập socket của từng phòng, và `user_sockets[user_id]` là mọi socket đang mở của một người. Nhờ vậy:

- Tin nhắn được phát tới mọi thành viên đang trực tuyến trong phòng và đồng thời được lưu bền vào cơ sở dữ liệu.
- Người dùng mở nhiều tab vẫn được tính là một trạng thái; chỉ khi socket cuối cùng đóng thì `is_online` mới chuyển thành false và `last_seen` được ghi lại, đồng thời sự kiện `user_status` được phát tới phòng.
- Socket chết (gửi lỗi) được dọn tự động khi phát tin.
- Phía giao diện gửi `ping` mỗi 25 giây để giữ kết nối qua reverse proxy của Render.

7.3. Dự phòng bằng REST

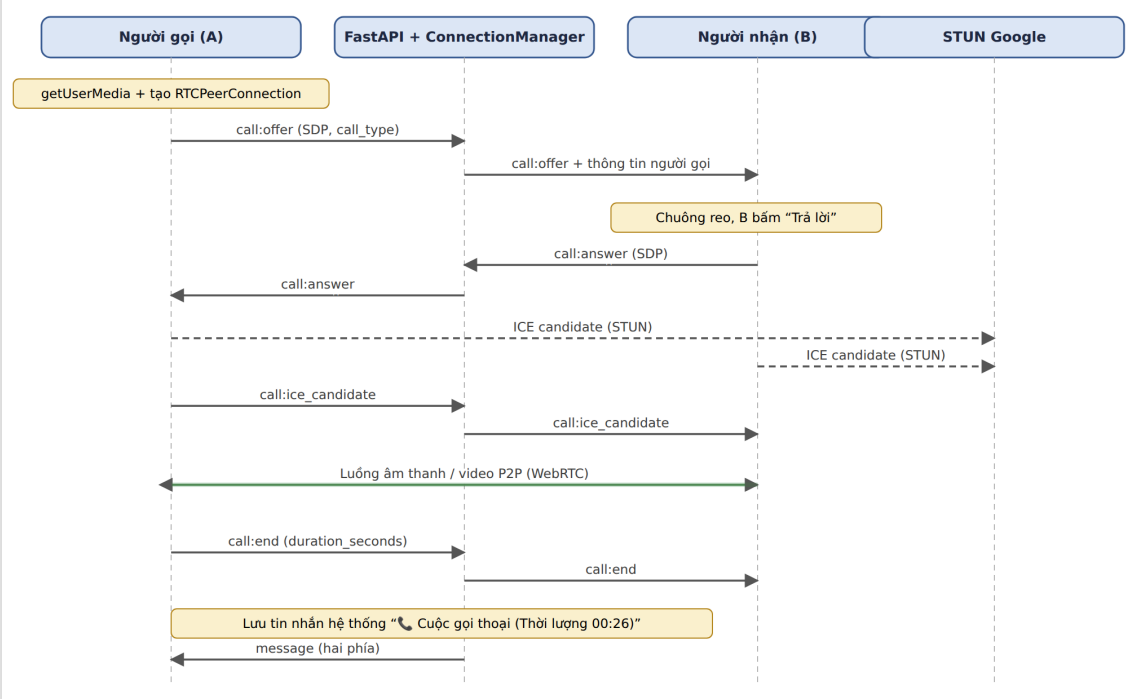
Ngoài WebSocket, tin nhắn có thể gửi bằng `POST /api/conversations/{id}/messages` khi kết nối mạng chập chờn. Tin nhắn mới tạo thông báo cho người nhận; khi người nhận mở cuộc trò chuyện, các tin chưa đọc được đánh dấu bằng `PUT /api/messages/{id}/read` và số lượng chưa đọc ở danh sách hội thoại được cập nhật.

CHƯƠNG 8. GỌI THOẠI VÀ GỌI VIDEO WEBRTC

Phần do Đinh Tuấn Quang phụ trách.

8.1. Luồng báo hiệu

Backend không xử lý media mà chỉ chuyển tiếp các bản tin báo hiệu giữa hai người. Hàm `_handle_call_signaling` nhận các sự kiện `call:offer`, `call:answer`, `call:ice_candidate`, `call:reject`, `call:end`, tự bổ sung mã người nhận hoặc mã hội thoại nếu thiếu, và gửi tới mọi socket đang mở của người nhận (qua `/api/ws/user`) đồng thời tới phòng chat. Hình 5 mô tả một cuộc gọi hoàn chỉnh.



Hình 5. Luồng cuộc gọi WebRTC và ghi lịch sử

8.2. Các cơ chế đảm bảo

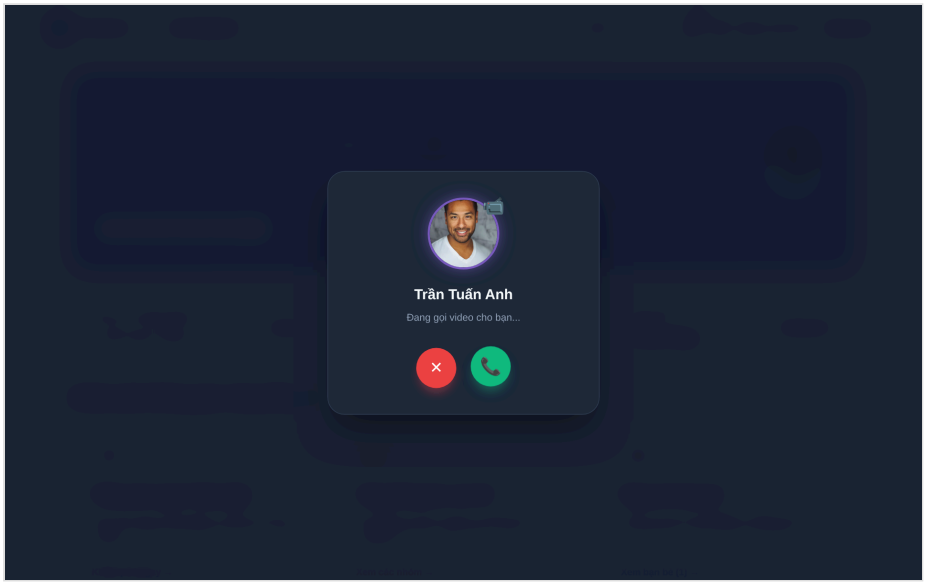
- **Hai kênh tín hiệu song song**: tín hiệu đi cả qua socket toàn cục và socket phòng chat để giảm nguy cơ mất bản tin khi người dùng đổi trang.
- **Người nhận ngoại tuyến**: nếu người nhận không có socket nào, người gọi nhận `call:unavailable`.
- **Chặn**: nếu hội thoại bị chặn, người gọi nhận `call:error` và không có cuộc gọi nào được tạo.
- **Hết thời gian chờ**: cuộc gọi tự huỷ sau 30 giây đổ chuông mà không có người nghe.
- **Ghi vết lịch sử**: khi nhận `call:end` hoặc `call:reject`, backend lưu một tin nhắn hệ thống vào hội thoại, ví dụ "📞 Cuộc gọi thoại (Thời lượng: 00:26)",

"📹 Cuộc gọi bị từ chối", "Đối phương đang bận" hoặc "Cuộc gọi nhỡ", rồi phát cho cả hai bên.

- **Âm thanh chuông**: tiếng đổ chuông và chuông reo được tổng hợp bằng Web Audio API (`OscillatorNode`), không cần tệp âm thanh ngoài.

8.3. Giao diện cuộc gọi

Thành phần `CallModal` hiển thị lớp phủ tối; cuộc gọi video có video đối phương toàn màn hình và video của mình thu nhỏ ở góc, cuộc gọi thoại hiển thị ảnh đại diện với hiệu ứng sóng. Thanh điều khiển có nút bật hoặc tắt micro, bật hoặc tắt camera, kết thúc cuộc gọi và đồng hồ đếm thời gian. Hình 6 là cửa sổ cuộc gọi đến ở máy người nhận khi chạy ứng dụng thật.



Hình 6. Cuộc gọi video đến ở máy người nhận

Hệ thống chỉ cấu hình máy chủ STUN công khai của Google (`stun:stun.l.google.com:19302`) mà chưa có máy chủ TURN; vì vậy cuộc gọi có thể không thiết lập được khi cả hai người cùng nằm sau NAT đối xứng (mục 12.3).

CHƯƠNG 9. BẢNG TIN, THÔNG BÁO VÀ KIỂM DUYỆT

Phần do Nguyễn Duy Long phụ trách.

9.1. Bảng tin cộng đồng

Người dùng đăng bài kèm danh mục (ví dụ Công nghệ, Đọc sách, Nghệ thuật, Sinh viên), lọc theo danh mục, xem chi tiết bài viết cùng bình luận theo thời gian, thả hoặc bỏ thả tim. Chỉ tác giả hoặc admin được xoá bài. Mỗi lượt thích và bình luận mới tạo thông báo cho tác giả. Trang Cộng đồng hiển thị các nhóm chủ đề do giao diện định nghĩa sẵn; số thành viên trong các thẻ này là dữ liệu tĩnh của giao diện, chưa được lấy từ cơ sở dữ liệu.

9.2. Thông báo và trạng thái hoạt động

Thông báo được tạo tự động khi có lời mời kết bạn, lời mời được chấp nhận, tin nhắn mới, bài viết được thích hoặc có bình luận mới. Thanh điều hướng có chuông thông báo kèm số chưa đọc; người dùng đánh dấu đọc từng thông báo hoặc đọc tất cả. Mọi thao tác đều kiểm tra quyền sở hữu: đánh dấu thông báo của người khác trả về 403. Trạng thái trực tuyến và thời điểm hoạt động gần nhất lấy từ `ConnectionManager` (mục 7.2).

9.3. Chặn, báo cáo và bảng điều khiển quản trị

Chặn là quan hệ hai chiều: người bị chặn không còn xuất hiện ở trang Khám phá của người chặn và ngược lại, không thể gửi lời mời hay nhắn tin, và các nút gọi bị vô hiệu hoá. Báo cáo gồm lý do (Spam, Quấy rối, Lừa đảo, Khác) và mô tả chi tiết. Bảng điều khiển quản trị gồm bốn chỉ số (tổng người dùng, đang hoạt động, bị khoá, báo cáo chờ duyệt), danh sách người dùng có tìm kiếm và lọc vai trò kèm khoá hoặc mở khoá tài khoản tức thì, và hàng đợi báo cáo cần xử lý.

CHƯƠNG 10. GIAO DIỆN REACT

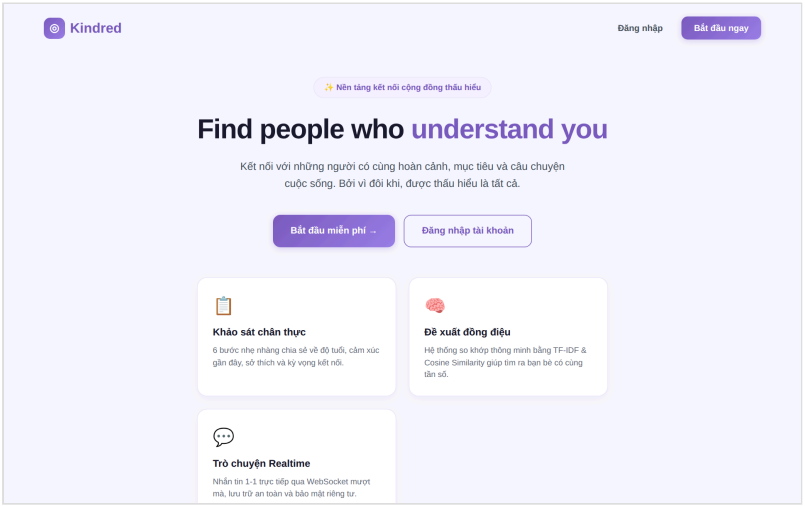
Phần do Lê Anh Tú phụ trách.

10.1. Các chức năng chính

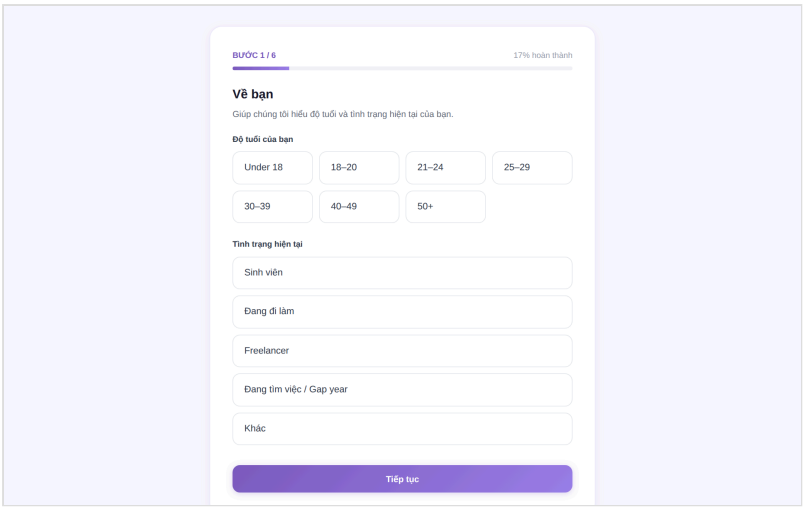
Giao diện là ứng dụng một trang xây bằng React 18 và Vite, dùng React Router v6, Context API cho trạng thái toàn cục và CSS thuần với bộ biến thiết kế (design tokens); không dùng TypeScript hay thư viện quản lý trạng thái ngoài. Các trang yêu cầu đăng nhập được bọc bởi `ProtectedRoute`. `AuthContext` giữ token và người dùng hiện tại, `CallContext` duy trì WebSocket toàn cục, nhận cuộc gọi đến ở mọi trang và điều khiển `CallModal`.

Bảng 7. Các trang giao diện

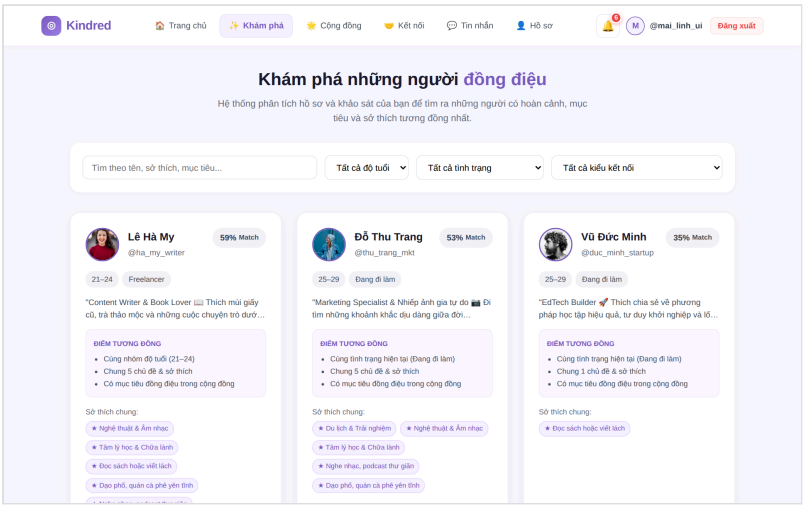
Trang	Chức năng
Landing, Login, Register	Giới thiệu, đăng nhập (kèm nút Google), đăng ký
Survey	Khảo sát onboarding 6 nhóm nội dung
Home	Tổng quan và lối vào các tính năng
Discover	Danh sách người đồng điệu có % Match, lý do, bộ lọc; nút Đồng ý hoặc Nhắn tin ngay trên thẻ
Connections	Bạn bè của tôi, Lời mời đã nhận, Lời mời đã gửi (liên kết trực tiếp ?`tab=received`)
Messages	Danh sách hội thoại, khung chat thời gian thực, nút gọi thoại và gọi video, chặn và báo cáo
Communities	Các nhóm chủ đề, bài viết, thích và bình luận
Profile	Xem và chỉnh sửa hồ sơ cá nhân
AdminDashboard	Thống kê, quản lý người dùng, kiểm duyệt báo cáo (chỉ admin)



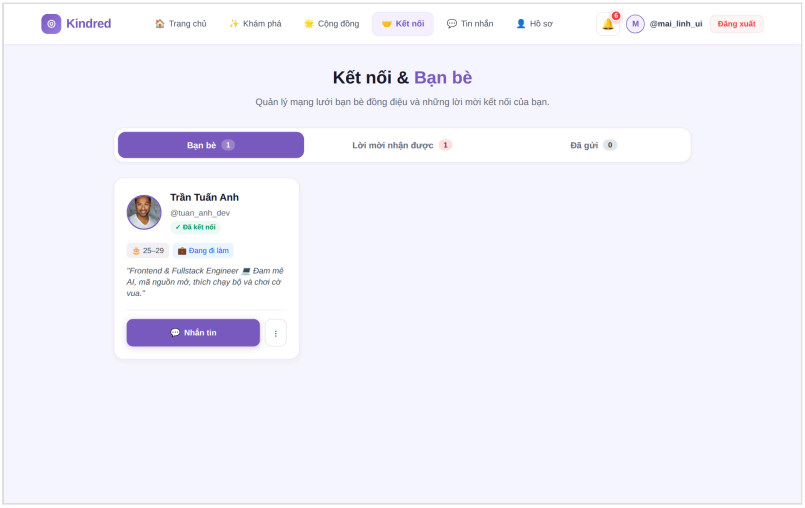
Hình 7. Trang chủ giới thiệu



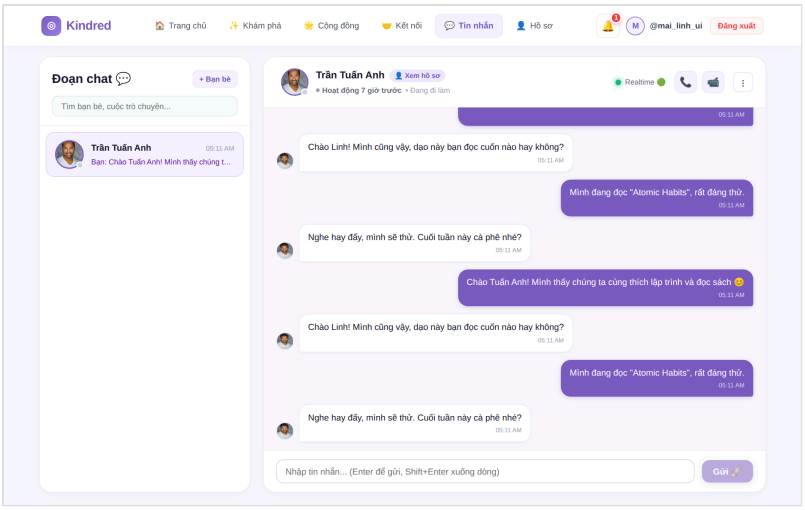
Hình 8. Khảo sát onboarding dành cho tài khoản mới



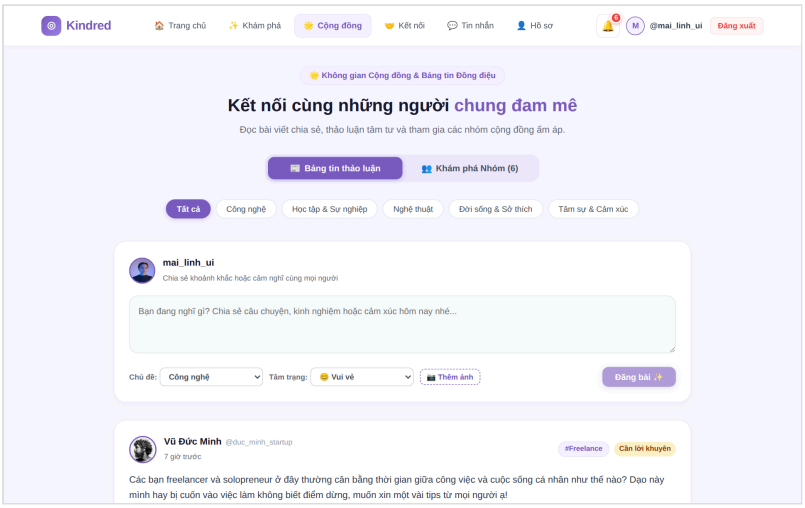
Hình 9. Trang Khám phá với % Match và lý do đồng điệu



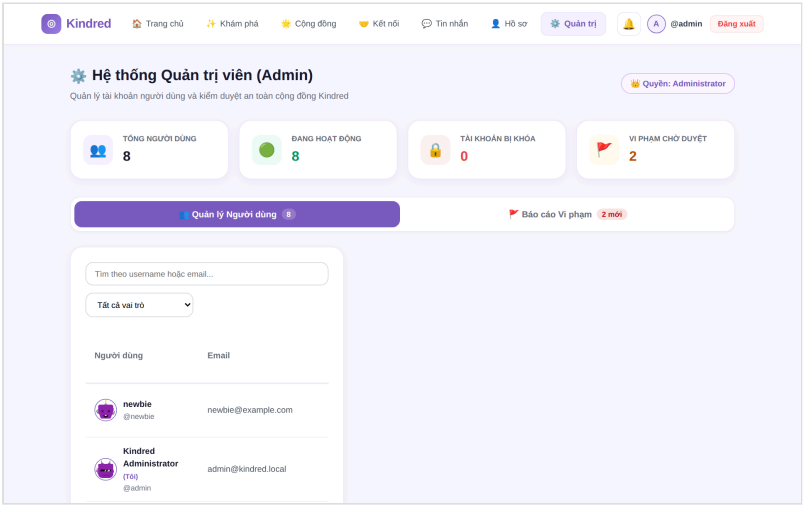
Hình 10. Quản lý kết nối



Hình 11. Nhắn tin thời gian thực



Hình 12. Trang Cộng đồng



Hình 13. Bảng điều khiển quản trị

10.2. Cách giao diện tương tác với backend

Bảng 8. Các điểm giao tiếp của giao diện

Thành phần	Cách giao diện tương tác
REST API	Axios dùng chung một instance; interceptor gắn JWT vào mọi request và xoá token khi nhận 401
WebSocket toàn cục	`CallContext` kết nối `/api/ws/user`, tự kết nối lại và gửi `ping` mỗi 25 giây
WebSocket phòng chat	Trang Messages kết nối `/api/ws/chat/{id}`; nếu lỗi sẽ gửi tin qua REST
WebRTC	`CallModal` tạo `RTCPeerConnection`, lấy micro và camera bằng `getUserMedia`, trao đổi SDP và ICE qua WebSocket
Đăng nhập Google	`GoogleLoginButton` dùng Google Identity Services, gửi ID token tới `/api/auth/google`
Cấu hình	Địa chỉ API đọc từ biến môi trường Vite `VITE_API_URL`; `vercel.json` cấu hình rewrite cho SPA

CHƯƠNG 11. KIỂM THỬ VÀ TRIỂN KHAI

Phần do Nguyễn Duy Long phụ trách.

11.1. Kiểm thử tự động

Backend có 8 script kiểm thử trong thư mục `backend/tests`, chạy trực tiếp bằng Python trên cơ sở dữ liệu SQLite. Nhóm đã chạy lại toàn bộ trước khi viết báo cáo và cả 8 script đều thành công (bảng 9). Riêng `test_full_suite.py` cần cơ sở dữ liệu đã có sẵn các bảng, nên trên một thư mục sạch phải chạy ứng dụng hoặc một script khác trước để tạo bảng.

Bảng 9. Các bộ kiểm thử backend

Script	Phạm vi kiểm thử	Kết quả
`test_full_suite.py`	7 nhóm: xác thực và bảo mật, hồ sơ, khảo sát, ghép đôi, kết nối, trò chuyện, thông báo và trạng thái online	Đạt 7/7
`test_connections.py`	Gửi, chấp nhận, từ chối lời mời kết nối	Đạt
`test_chat_flow.py`	Dịch vụ chat: tạo hội thoại, gửi, đọc, phân quyền	Đạt
`test_realtime_ws.py`	WebSocket chat độc lập	Đạt
`test_posts_and_feed.py`	Bài viết, thích, bình luận, phân quyền xoá	Đạt
`test_block_and_report.py`	Chặn hai chiều và báo cáo vi phạm	Đạt
`test_admin.py`	Thống kê, khoá tài khoản, duyệt báo cáo, kiểm tra quyền	Đạt
`test_notifications_and_status.py`	Thông báo và trạng thái online, last_seen	Đạt

Phần giao diện chưa có kiểm thử tự động; nhóm kiểm tra bằng cách chạy lệnh `npm run build` (đóng gói thành công, tệp JavaScript khoảng 413 kB, gzip khoảng 115 kB) và thao tác thủ công trên trình duyệt, kể cả cuộc gọi giữa hai tài khoản trên cùng máy theo hướng dẫn trong README.

11.2. Triển khai

Bảng 10. Hạ tầng triển khai

Thành phần	Nền tảng	Ghi chú
Frontend	Vercel	Tự triển khai khi đẩy mã lên nhánh `main`; `vercel.json` rewrite mọi đường dẫn về `index.html`
Backend	Render	Lệnh chạy `uvicorn app.main:app --host 0.0.0.0 --port $PORT`; hỗ trợ WebSocket
Cơ sở dữ liệu	Supabase	PostgreSQL, kết nối qua `DATABASE_URL`

Các biến môi trường của backend gồm `DATABASE_URL`, `JWT_SECRET_KEY` (chuỗi ngẫu nhiên tối thiểu 32 ký tự), `JWT_ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `GOOGLE_CLIENT_ID` và `CORS_ORIGINS`. Do trạng thái WebSocket nằm trong bộ nhớ tiến trình, backend phải chạy đúng một instance; muốn mở rộng nhiều instance cần thêm một kho chia sẻ như Redis.

CHƯƠNG 12. CÔNG NGHỆ VÀ KẾT LUẬN

12.1. Công nghệ và thư viện

Bảng 11. Thư viện chính

Nhóm	Thư viện (phiên bản)	Vai trò
Web backend	fastapi 0.115.0, uvicorn 0.30.6	REST API, WebSocket, máy chủ ASGI
Cơ sở dữ liệu	sqlalchemy 2.0.36, psycopg2-binary 2.9.10	ORM, driver PostgreSQL
Bảo mật	bcrypt 4.2.0, pyjwt 2.13.0, google-auth 2.35.0	Băm mật khẩu, JWT, xác minh Google
Ghép đôi	scikit-learn 1.9.0	TfidfVectorizer, cosine_similarity
Khác	python-dotenv 1.2.3, requests 2.32.3	Biến môi trường, gọi HTTP
Frontend	react 18.3.1, react-router-dom 6.26.2, axios 1.7.7, vite 5.4.2	Giao diện, định tuyến, gọi API, build
Thời gian thực	WebSocket gốc, WebRTC, Web Audio API	Chat, cuộc gọi, âm thanh chuông

12.2. Kết quả đạt được

- Hoàn thành ứng dụng nhiều người dùng gồm khảo sát, ghép đôi có giải thích, kết nối, nhắn tin thời gian thực, gọi thoại và video, bảng tin, thông báo, chặn, báo cáo và quản trị.
- Thuật toán ghép đôi chỉ dùng TF-IDF, cosine và quy tắc nên chạy nhanh, kết quả giải thích được và không phụ thuộc dịch vụ trả phí.
- Backend phân tầng rõ, có 8 script kiểm thử đều thành công; hệ thống đã triển khai trên Vercel, Render và Supabase.

12.3. Hạn chế

- Chưa đánh giá chất lượng ghép đôi bằng dữ liệu người dùng thật; điểm số mới được kiểm tra trên 6 tài khoản mẫu.
- TF-IDF được tính lại trên toàn bộ ứng viên ở mỗi request nên chi phí tăng tuyến tính theo số người dùng; chưa có bộ nhớ đệm.
- Trạng thái WebSocket nằm trong bộ nhớ nên chỉ chạy được một instance; chưa có TURN nên cuộc gọi có thể thất bại sau NAT đối xứng.
- Cơ sở dữ liệu tạo bằng `create_all` và vài lệnh `ALTER TABLE` thủ công, chưa có công cụ migration như Alembic.
- Danh sách nhóm cộng đồng là dữ liệu tĩnh ở giao diện; tài khoản quản trị mặc định phải được đổi mật khẩu qua biến môi trường khi triển khai thật.

- Giao diện chưa có kiểm thử tự động và các script backend chưa chạy dưới pytest hay CI.

12.4. Hướng phát triển

- Dùng Redis cho trạng thái WebSocket để mở rộng nhiều instance; bổ sung máy chủ TURN cho cuộc gọi.
- Đưa cộng đồng vào cơ sở dữ liệu (tạo, tham gia, kiểm duyệt nhóm) và gợi ý cộng đồng theo khảo sát.
- Thêm Alembic cho migration, pytest cùng CI cho cả backend và giao diện.
- Cải thiện ghép đôi bằng nhúng ngữ nghĩa tiếng Việt, kèm bộ dữ liệu đánh giá và phản hồi “phù hợp hoặc không” của người dùng.

TÀI LIỆU THAM KHẢO

[1] Pedregosa F. et al., “Scikit-learn: Machine Learning in Python”, Journal of Machine Learning Research, 2011.

[2] Salton G., Buckley C., “Term-weighting approaches in automatic text retrieval”, Information Processing & Management, 1988.

[3] Fette I., Melnikov A., “The WebSocket Protocol”, RFC 6455, IETF, 2011.

[4] Jones M., Bradley J., Sakimura N., “JSON Web Token (JWT)”, RFC 7519, IETF, 2015.

[5] W3C, “WebRTC 1.0: Real-Time Communication Between Browsers”, https://www.w3.org/TR/webrtc/

[6] Tài liệu FastAPI: https://fastapi.tiangolo.com/ ; SQLAlchemy: https://docs.sqlalchemy.org/

[7] Tài liệu React: https://react.dev/ ; Vite: https://vitejs.dev/

[8] Google Identity Services: https://developers.google.com/identity/gsi/web